

SKILL_WEEK - 12 (SESSION NUMBER - 23)

Task 1: To execute large pipelines, launch multiple jobs, measure stability, monitor resources, detect failures, and analyze performance.

```
gettimeofday(&end, NULL);
getrusage(RUSAGE_CHILDREN, &usage);

double elapsed =
    (end.tv_sec - start.tv_sec) +
    (end.tv_usec - start.tv_usec) / 1000000.0;

printf("\nPipeline Processes:\n");
printf("Job 1 PID: %d\n", p1);
printf("Job 2 PID: %d\n", p2);
printf("Job 3 PID: %d\n", p3);

if (WIFEXITED(s1) && WEXITSTATUS(s1) == 0 &&
    WIFEXITED(s2) && WEXITSTATUS(s2) == 0 &&
    WIFEXITED(s3) && WEXITSTATUS(s3) == 0)
{
    printf("Pipeline completed without failures.\n");
}
else
{
    printf("Pipeline failure detected.\n");
}

printf("Execution Time: %.6f seconds\n", elapsed);

printf("Child User CPU Time: %ld.%06d seconds\n",
    usage.ru_utime.tv_sec,
    usage.ru_utime.tv_usec);

printf("Child System CPU Time: %ld.%06d seconds\n",
    usage.ru_stime.tv_sec,
    usage.ru_stime.tv_usec);

printf("Performance analysis completed successfully.\n");

return 0;
```

OUTPUT:

```
[amrutha@Amruthas-MacBook-Air OSSP % gcc src/session23_task1.c -o bin/session23_task1
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/session23_task1
3

Pipeline Processes:
Job 1 PID: 28461
Job 2 PID: 28462
Job 3 PID: 28463
Pipeline completed without failures.
Execution Time: 0.008998 seconds
Child User CPU Time: 0.003448 seconds
Child System CPU Time: 0.010428 seconds
Performance analysis completed successfully.
amrutha@Amruthas-MacBook-Air OSSP %
```

Task 2: To document architecture, create flow diagrams, explain execution paths, describe system calls, record design decisions, and review documentation.

```
## Execution Path

1. User enters a command.
2. Shell reads the command.
3. Parser splits the command into tokens.
4. Syntax is validated.
5. Shell determines whether the command is built-in or external.
6. External commands create a child process using fork().
7. Child executes the command using exec().
8. Parent waits using waitpid().
9. Pipes, redirection and signals are handled when required.
10. Shell returns to the command prompt.

## Important System Calls

fork()    - Creates a child process.
exec()    - Executes a new program.
waitpid() - Waits for child process completion.
pipe()    - Creates communication between processes.
dup2()    - Redirects input or output.
open()    - Opens files.
close()   - Closes file descriptors.
kill()    - Sends signals to processes.

## Design Decisions

- Modular structure is used for easier maintenance.
- fork() and exec() are used for external command execution.
- Pipes connect multiple processes.
- dup2() handles input and output redirection.
- Signal handling provides process control.
- Errors are handled without terminating the shell unexpectedly.

## Review

The architecture, execution flow, system calls and design
decisions were documented and reviewed successfully.
```

OUTPUT:

```
[amrutha@Amruthas-MacBook-Air OSSP % nano docs/session23_task2.md
[amrutha@Amruthas-MacBook-Air OSSP % cat docs/session23_task2.md
# Shell Architecture Documentation

## Architecture

User Input
  |
  v
Main Shell Loop
  |
  v
Parser
  |
  v
Command Executor
  |
  +---- Built-in Commands
  |
  +---- External Commands
           |
           v
         fork()
           |
           v
        exec()
           |
           v
      waitpid()
```

(SESSION NUMBER - 24)

Task 1: To organize source code, improve build scripts, add README, verify repository structure, tag releases, and validate deliverables.

```
#!/bin/bash

echo "OSSP Project Build"
echo "-----"

mkdir -p bin
mkdir -p obj

for file in src/*.c
do
    name=$(basename "$file" .c)

    if grep -q "int main" "$file"; then
        echo "Building executable: $name"

        gcc "$file" -o "bin/$name"

        if [ $? -eq 0 ]; then
            echo "$name : SUCCESS"
        else
            echo "$name : FAILED"
        fi
    else
        echo "Building module: $name"

        gcc -c "$file" -o "obj/$name.o"

        if [ $? -eq 0 ]; then
            echo "$name module : SUCCESS"
        else
            echo "$name module : FAILED"
        fi
    fi
done

echo
echo "Build process completed."
```

OUTPUT:

```
[amrutha@Amruthas-MacBook-Air OSSP % nano scripts/build_all.sh
amrutha@Amruthas-MacBook-Air OSSP % chmod +x scripts/build_all.sh
./scripts/build_all.sh
OSSP Project Build
-----
Building module: builtin
builtin module : SUCCESS
Building executable: double_quote_parser
double_quote_parser : SUCCESS
Building executable: escape_parser
escape_parser : SUCCESS
Building module: executor
executor module : SUCCESS
Building executable: file_copy
file_copy : SUCCESS
Building executable: main
main : SUCCESS
Building executable: parser
parser : SUCCESS
```

Task 2: To create demo scenarios, validate features, rehearse presentation, demonstrate pipelines, demonstrate signals, and perform final project review.

```
#!/bin/bash

echo "=====
echo "      OSSP FINAL PROJECT DEMO
echo "=====

echo
echo "1. Testing Pipeline Feature"
echo "-----"
echo "Operating Systems" > demo_input.txt
echo "Linux Programming" >> demo_input.txt
echo "Operating Systems Project" >> demo_input.txt

cat demo_input.txt | grep "Operating" | wc -l

echo
echo "Pipeline demonstrated successfully."

echo
echo "2. Testing File Redirection"
echo "-----"
echo "OSSP Redirection Test" > demo_output.txt
cat demo_output.txt

echo
echo "Redirection demonstrated successfully."

echo
echo "3. Testing Process Information"
echo "-----"
echo "Shell PID: $$"
echo "Process Group:"
ps -o pid,ppid,pgid,command -p $$

echo
echo "4. Validating Project Structure"
echo "-----"
echo "Source Directory:"
ls src | head

echo
echo "Binary Directory:"
ls bin | head

echo
echo "5. Final Review"
echo "-----"
echo "Pipeline Feature : PASS"
echo "Redirection      : PASS"
echo "Process Control  : PASS"
echo "Project Structure: PASS"

echo
echo "Final project demo completed successfully."
```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air OSSP % chmod +x scripts/final_demo.sh
amrutha@Amruthas-MacBook-Air OSSP % ./scripts/final_demo.sh
=====
      OSSP FINAL PROJECT DEMO
=====

1. Testing Pipeline Feature
-----
      2

Pipeline demonstrated successfully.

2. Testing File Redirection
-----
OSSP Redirection Test

Redirection demonstrated successfully.

3. Testing Process Information
-----
Shell PID: 31510
Process Group:
  PID  PPID  PGID COMMAND
31510  99507  31510 /bin/bash ./scripts/final_demo.sh
```

```
4. Validating Project Structure
-----
Source Directory:
builtin.c
double_quote_parser.c
escape_parser.c
executor.c
file_copy.c
main.c
parser.c
parser.c.save
process_launcher.c
process_states.c

Binary Directory:
double_quote_parser
escape_parser
file_copy
main
my_shell
parser
process_launcher
process_states
quote_parser
session11_task1

5. Final Review

Pipeline Feature : PASS
Redirection      : PASS
Process Control  : PASS
Project Structure: PASS

Final project demo completed successfully.
```